PROMETEO

thePower
FP Oficial

Sistema web de consultas al ERP de TECSOLED

2025-2026 · Desarrollo de Aplicaciones Web (Online)

Michael Félix Gutiérrez Mejía

Javier Arnedo Torres

Tutor: Ignacio Poveda Salinas

DEDICATORIA

A nuestras familias, que soportaron pacientemente las tardes y los fines de semana delante del ordenador.

A los compañeros de TECSOLED, que sin saberlo fueron los primeros usuarios de esta aplicación y cuyas quejas sobre el ERP fueron la mejor documentación de requisitos que podíamos tener.

ÍNDICE DE CONTENIDOS

DEDICATORIA 2
ABSTRACT 4
JUSTIFICACIÓN DEL PROYECTO 5
Motivación 5
Estado de la cuestión: alternativas analizadas 5
INTRODUCCIÓN 6
OBJETIVOS 7
DESCRIPCIÓN 9
Arquitectura de la solución 9
Estructura del proyecto 9
Endpoints de la API 9
Casos de uso 10
Diagrama E/R 13
Diagrama de flujo de navegación 14
Interfaces 14
TECNOLOGÍA 14
Stack tecnológico 14
Por que CSS propio en lugar de Bootstrap o Tailwind 15
METODOLOGÍA 17
Metodología de trabajo 17
Reparto de trabajo 17
Planificación en fases 17
Presupuesto 18

README y control de versiones 18
TRABAJOS FUTUROS 18
Pendiente para la entrega final (junio 2026) 18
Mejoras y ampliaciones que nos gustaría implementar 18
CONCLUSIONES 20
Valoración técnica 20
Aprendizajes 20
Valoración personal 20
REFERENCIAS 21

ABSTRACT

TECSOLED S.L. es una empresa de distribución e instalación de iluminación LED. Su día a día incluye decenas de consultas sobre stock, precios y composición de artículos, todas ellas a través de un ERP que no está pensado para ese uso: obtener un dato básico puede costar entre 30 y 45 segundos de navegación por varias pantallas.

Este proyecto presenta el diseño y desarrollo de una aplicación web interna que resuelve ese problema sin modificar el ERP. Consiste en una API REST en Node.js con Express, que consulta en modo lectura la base de datos SQL Server del ERP, y un frontend en HTML5, CSS3 y JavaScript puro accesible desde cualquier dispositivo de la red corporativa.

El sistema permite buscar productos con autocompletado, consultar stock por ubicación física, ver el despiece de cualquier artículo y acceder a un panel de facturación con gráficos mensuales. El tiempo de respuesta medido es inferior a 800 ms, frente a los 30-45 segundos del ERP.

Palabras clave: Node.js, Express, JWT, SQL Server, HTML5, CSS3, ERP, stock, facturación.

JUSTIFICACIÓN DEL PROYECTO

Motivación

Michael trabaja en TECSOLED desde antes de empezar el ciclo formativo. Llevaba tiempo viendo cómo cualquier consulta básica (¿cuántas unidades hay en almacén?, ¿cuál es el PVP?) acababa convirtiéndose en un rato perdido navegando por el ERP. Cuando llegó el momento de elegir el TFG, lo tuvo claro: algo que se usara de verdad en la empresa. Midió el tiempo medio de una consulta en el ERP: entre 30 y 45 segundos. Multiplicado por las decenas de consultas diarias de todos los departamentos, el impacto es considerable.

Javier se incorporó al proyecto con experiencia en diseño web y ganas de afrontar un proyecto real con usuarios finales. Entre los dos definimos los requisitos, repartimos el trabajo y acordamos que la solución tenía que ser sencilla de usar sin formación adicional.

Estado de la cuestión: alternativas analizadas

Antes de ponernos a programar, miramos qué opciones había para resolver este problema:

Alternativa	Descripción	Por que la descartamos
Módulos adicionales del ERP	El proveedor ofrece módulos extra para consultas rápidas.	Coste elevado, tiempo de implantación largo y dependencia total del proveedor.
Migrar a un ERP moderno (Odoo, SAP)	Cambiar el sistema por uno con mejor interfaz.	Proyecto de meses o años, riesgo operativo alto y coste inasumible para la empresa.
Herramientas BI (Power BI, Tableau)	Plataformas de análisis conectadas al SQL Server.	Orientadas al análisis histórico, no a la consulta rápida en tiempo real. Requieren formación.
Aplicación propia (este TFG)	Capa web ligera sobre la BD del ERP, solo lectura.	Solución elegida: sin coste de licencias, sin modificar el ERP y lista en semanas.

La aplicación propia era la única opción que se adaptaba exactamente a lo que necesitaba TECSOLED: rápida de desarrollar, sin coste adicional y sin tocar nada del sistema que ya funcionaba.

INTRODUCCIÓN

La aplicación web TECSOLED ERP API permite a los empleados hacer en menos de 5 segundos las consultas que ahora les cuestan entre 30 y 45 segundos en el ERP. Para hacerse una idea del salto:

Consulta	Tiempo en el ERP	Tiempo con la app
Stock disponible de un artículo	30-45 segundos	menos de 1 segundo
PVP y descripción	30-45 segundos	menos de 1 segundo
Ubicaciones físicas en almacén	45-60 segundos	menos de 1 segundo
Despiece de un artículo	60-90 segundos	menos de 2 segundos
KPIs de facturación del mes	2-3 minutos	menos de 2 segundos

Las funcionalidades principales de la aplicación son:

Búsqueda de productos: el empleado escribe parte de la referencia o el nombre y aparecen resultados en tiempo real, sin necesidad de buscar pantalla por pantalla.

Detalle del artículo: stock en almacén, PVP, descripción técnica, stock mínimo y unidades pendientes de recibir, todo en una sola pantalla.

Ubicaciones físicas: en qué almacén, subalmacén y estantería exacta se encuentra cada unidad del artículo consultado.

Despiece: lista de componentes que forman el artículo terminado y disponibilidad de cada uno en almacén.

Panel de facturación (solo para administración): KPIs del mes, trimestre o año, gráfico de evolución mensual y comparativa con el período anterior.

El sistema no modifica ningún dato del ERP en ningún momento. Todas las consultas sobre productos y facturación son de solo lectura. La gestión de usuarios de la aplicación se hace en una base de datos SQLite independiente.

OBJETIVOS

El objetivo principal del proyecto es reducir el tiempo de consulta al ERP de TECSOLED de 30-45 segundos a menos de 5 segundos, sin modificar el sistema existente y sin coste adicional de licencias.

De ese objetivo principal se derivan cinco objetivos específicos:

1. Implementar una API REST en Node.js y Express con acceso de solo lectura al SQL Server del ERP.
2. Desarrollar un frontend en HTML5, CSS3 y JavaScript sin frameworks, que funcione en ordenadores y móviles.
3. Implementar autenticación con JWT y dos roles: comercial/almacén y administración.
4. Conseguir tiempos de respuesta inferiores a 800 ms en red local.
5. Desplegar la aplicación con PM2 en el servidor interno de TECSOLED.

RFTP: Requisitos, Funciones, Tareas y Pruebas

R01: El sistema solo permite acceder a empleados autorizados de TECSOLED.

- R01F01: Validar las credenciales del usuario antes de dar acceso.
  - R01F01T01: Crear la base de datos SQLite de usuarios e implementar POST /api/auth/login con validación bcryptjs.
    - R01F01T01P01: Login con credenciales correctas devuelve 200 y token JWT.
  - R01F01T02: Implementar el middleware que verifica el JWT en cada ruta protegida.
    - R01F01T02P01: Petición sin token a ruta protegida devuelve 401.
- R01F02: El sistema diferencia entre rol 1 (comercial/almacén) y rol 2 (administración).
  - R01F02T01: Incluir el campo rol en el JWT y validarlo con un middleware de autorización.
    - R01F02T01P01: Acceso al dashboard con rol 1 devuelve 403.

R02: El sistema permite buscar productos del ERP de forma rápida.

- R02F01: Búsqueda por referencia o nombre con autocompletado en tiempo real.
  - R02F01T01: Implementar GET /api/productos/buscar con consulta sobre SQL Server.
    - R02F01T01P01: Buscar "LED" devuelve resultados en menos de 800 ms.
  - R02F01T02: Debounce de 300 ms en el frontend para no saturar la API.
    - R02F01T02P01: Escritura rápida lanza solo una petición al dejar de escribir.

R03: El sistema muestra información detallada de cada producto.

- R03F01: Mostrar stock, PVP, descripción, ubicaciones físicas y despiece.
  - R03F01T01: Crear GET /api/productos/:ref, /ubicaciones y /despiece.
    - R03F01T01P01: Detalle de un producto muestra los tres bloques de información.

R03F01T02: Cargar los tres bloques en paralelo con Promise.all.

R03F01T02P01: Las tres peticiones se lanzan simultáneamente (verificado en Network tab).

R04: El sistema muestra un panel de facturación al rol de administración.

R04F01: KPIs de ventas y gráfico de evolución mensual.

R04F01T01: Crear GET /api/facturación/dashboard, /evolución y /comparativa.

R04F01T01P01: Dashboard con rol 2 muestra datos y gráfico correctos.

R04F01T02: Integrar Chart.js via CDN para el gráfico de barras.

R04F01T02P01: Gráfico visible y correcto en Chrome, Firefox y Edge.

R05: El sistema está disponible de forma continua en la red corporativa.

R05F01: Despliegue en servidor interno con PM2 y arranque automático.

R05F01T01: Crear ecosystem.config.js y configurar pm2 startup.

R05F01T01P01: Reinicio del servidor, la aplicación arranca sola.

R05F01T02: Variables de entorno con dotenv para separar credenciales del código.

R05F01T02P01: El repositorio no contiene credenciales. El .env está en .gitignore.

DESCRIPCIÓN

Arquitectura de la solución

La aplicación sigue una arquitectura cliente-servidor de tres capas. El navegador del empleado actúa como cliente; el servidor Express sirve tanto la API como los archivos estáticos del frontend; y en la capa de datos conviven dos fuentes: el SQL Server del ERP (solo lectura) y una base de datos SQLite propia para los usuarios de la aplicación.

El flujo de una consulta típica es el siguiente: el empleado escribe en el buscador del navegador, el frontend lanza una petición HTTP con el token JWT en la cabecera, el middleware verifica el token, el servicio correspondiente ejecuta la query sobre SQL Server y el resultado vuelve como JSON al navegador en menos de 800 ms.

[INSERTAR: Diagrama de arquitectura]

Estructura del proyecto

El código está organizado en dos grandes bloques: el backend en la carpeta src/ y el frontend en la raiz. En src/ encontramos los controladores (lógica HTTP), los servicios (consultas SQL), las rutas y los middlewares (autenticación JWT y control de roles). En el frontend, cada página tiene su propio archivo JavaScript: productos.js para la búsqueda, producto-detalle.js para el detalle y dashboard.js para la facturación. El CSS está centralizado en un único archivo styles.css con variables personalizadas para toda la interfaz.

Endpoints de la API

La API expone 9 rutas. Todas devuelven JSON con el formato { success, data } en caso de éxito o { success: false, message } si hay algun error.

Metodo	Ruta	Descripción	Auth	Rol mínimo
POST	/api/auth/login	Login. Devuelve token JWT.	No	Todos
POST	/api/auth/cambiar-password	Cambio de contraseña del usuario.	Si	1
GET	/api/productos/buscar	Búsqueda por referencia o nombre (max. 20).	Si	1

GET	/api/productos/:ref	Datos generales del artículo.	Si	1
GET	/api/productos/:ref/ubicaciones	Stock por almacén y estantería.	Si	1
GET	/api/productos/:ref/despiece	Componentes y disponibilidad.	Si	1
GET	/api/facturación/dashboard	KPIs del período elegido.	Si	2
GET	/api/facturación/evolución	Evolución mensual del ano.	Si	2
GET	/api/facturación/comparativa	Variación respecto al período anterior.	Si	2

Casos de uso

CU-01: Iniciar sesión

Campo	Descripción
Actor	Cualquier empleado de TECSOLED.
Descripción	El usuario introduce su correo y contraseña para obtener un token JWT.
Precondición	El usuario existe en la BD SQLite y tiene la cuenta activa.
Flujo principal	1. Introduce correo y contraseña en login.html. 2. El frontend envía POST /api/auth/login. 3. El backend valida con bcryptjs y genera el JWT. 4. El token se guarda en sessionStorage y el usuario accede a la app.
Flujo alternativo	Credenciales incorrectas: mensaje de error, sin redirección.
Postcondición	El usuario tiene acceso a las funciones de su rol.
Datos de entrada	Correo electrónico y contraseña.
Datos de salida	Token JWT (éxito) o mensaje de error (fallo).

CU-02: Buscar producto

Campo	Descripción
Actor	Empleado con rol 1 o rol 2.
Descripción	El usuario escribe parte de una referencia o nombre y obtiene resultados en tiempo real.
Precondición	El usuario ha iniciado sesión y el token JWT es válido.
Flujo principal	1. Escribe en el campo de búsqueda (mínimo 2 caracteres). 2. Tras 300 ms sin escribir se lanza GET /api/productos/buscar. 3. Aparecen hasta 20 resultados con referencia y nombre. 4. El usuario hace clic en el que le interesa.
Flujo alternativo	Sin resultados: mensaje "No se encontraron productos". Token expirado: redirección al login.
Postcondición	El navegador abre la página de detalle del artículo.
Datos de entrada	Cadena de texto (referencia o nombre).
Datos de salida	Lista de artículos con referencia, nombre y familia.

CU-03: Ver detalle de producto

Campo	Descripción
Actor	Empleado con rol 1 o rol 2.
Descripción	El usuario consulta el stock, PVP, ubicaciones y despiece de un artículo.
Precondición	El usuario ha seleccionado un artículo en la búsqueda.
Flujo principal	1. Se carga producto.html con la referencia en la URL. 2. El frontend lanza tres peticiones en paralelo con Promise.all: · GET /api/productos/:ref (datos generales) · GET /api/productos/:ref/ubicaciones (estanterías) · GET /api/productos/:ref/despiece (componentes) 3. Los tres bloques se muestran al recibir las tres respuestas.
Flujo alternativo	Sin despiece: "Este artículo no tiene despiece registrado". Sin stock en ubicaciones: la tabla aparece vacía.

Postcondición	El empleado ve todos los datos del artículo en una sola pantalla.
Datos de entrada	Referencia del artículo (en la URL).
Datos de salida	Stock, PVP, descripción, tabla de ubicaciones y tabla de despiece.

CU-04: Consultar panel de facturación

Campo	Descripción
Actor	Empleado con rol 2 (administración).
Descripción	El usuario consulta los KPIs de ventas, el gráfico mensual y la comparativa con el período anterior.
Precondición	El usuario ha iniciado sesión con token JWT de rol 2.
Flujo principal	1. Accede a dashboard.html. 2. Se cargan los KPIs (GET /api/facturación/dashboard). 3. Se carga el gráfico de evolución y la comparativa. 4. El usuario puede cambiar el período (mes / trimestre / ano).
Flujo alternativo	Rol 1 intenta acceder: el backend devuelve 403 y el frontend redirige al login.
Postcondición	Los datos se actualizan al cambiar el período sin recargar la página.
Datos de entrada	Período seleccionado (mes, trimestre o ano).
Datos de salida	KPIs, gráfico de barras y variación porcentual.

DESCRIPCIÓN: Solicitar ayuda al especialista	
PRECONDICIONES: Usuario logado	**POSTCONDICIONES**: Solicitud en espera Se inicia el chat
DATOS ENTRADA	**DATOS SALIDA**

Nombre especialista Id usuario Id especialista	Nombre especialista Id usuario Id especialista Idchat Valoración fecha/hora
TABLAS: USUARIOS CHAT	**CLASES**: ESPECIALISTA.PHP USUARIO NORMAL.PHP CHAT.PHP
INTERFACES: PERFILUSUARIO.HTML CHAT.HTML	

DISEÑOS (Los que procedan según el tipo de proyecto)

Diagrama E/R

La aplicación no crea tablas en el ERP. Las consultas de productos y facturación son de solo lectura sobre las tablas existentes en SQL Server. La única base de datos propia es la SQLite que usamos para gestionar los usuarios de la aplicación. Sus campos son:

Campo	Tipo	Restricción	Descripción
id	INTEGER	PRIMARY KEY	Identificador único del usuario.
email	TEXT	UNIQUE NOT NULL	Correo electrónico (login).

password_hash	TEXT	NOT NULL	Hash bcrypt de la contraseña.
rol	INTEGER	NOT NULL	1 = comercial/almacén · 2 = administración.
activo	INTEGER	DEFAULT 1	0 = cuenta desactivada.

[INSERTAR: Diagrama E/R con las tablas del ERP consultadas y la tabla Usuarios]

Diagrama de flujo de navegación

Hay dos flujos según el rol del usuario. El rol 1 (comercial, almacén, compras) solo tiene acceso a la búsqueda y al detalle de producto. El rol 2 (administración) tiene acceso a todo lo anterior más el panel de facturación, al que puede acceder directamente desde la barra lateral.

[INSERTAR: Diagrama de flujo de navegación]

Interfaces

Javier diseño el sistema de estilos desde cero con CSS puro, usando variables personalizadas para colores, tipografía y bordes. El resultado es una interfaz oscura y limpia que encaja con la imagen industrial de TECSOLED. El diseño es responsive y funciona desde 480 px de ancho, aunque el uso principal se espera en ordenadores de escritorio.

[INSERTAR: Capturas de pantalla: login, búsqueda, detalle de producto, panel de facturación]

TECNOLOGÍA

Stack tecnológico

A continuación, se detallan todas las tecnologías y herramientas utilizadas en el proyecto:

Tecnología	Versión	Uso en el proyecto
Node.js	20 LTS	Runtime del servidor. Elegido por su ecosistema npm y su rendimiento asíncrono.
Express	4.18	Framework web para definir rutas, middlewares y gestionar errores de la API.
mssql	10.x	Driver de SQL Server. Gestiona el pool de conexiones al ERP (max. 10).

better-sqlite3	9.x	Base de datos SQLite síncrona para los usuarios de la aplicación.
jsonwebtoken	9.x	Generación y validación de tokens JWT (algoritmo HS256, expiración 8 horas).
bcryptjs	2.x	Hash de contraseñas con salt de coste 10.
express-validator	7.x	Validación de entradas en los endpoints de autenticación.
helmet	7.x	Cabeceras de seguridad HTTP. CSP parcialmente desactivada para Chart.js CDN.
cors	2.x	Control de origen. El dominio permitido se configura en .env.
winston	3.x	Logging en archivos con rotación automática (max. 5 MB por fichero).
dotenv	16.x	Variables de entorno desde el fichero .env.
PM2	5.x	Gestor de procesos para el despliegue. Reinicio automático tras caida.
HTML5 + CSS3	Nativo	Frontend: estructura y estilos. Sin frameworks ni preprocesadores.
JavaScript ES6+	Nativo	Frontend: lógica del cliente. Sin React, Vue ni jQuery.
Chart.js	4.x (CDN)	Gráfico de barras del panel de facturación.
SQL Server	2019	Base de datos del ERP. Solo lectura.
SQLite	3.x	Base de datos de usuarios de la aplicación.

Por que CSS propio en lugar de Bootstrap o Tailwind

Al principio Javier propuso usar Bootstrap para ir más rápido, pero lo descartamos por dos razones. La primera es que Bootstrap aniadia más de 200 KB de CSS del que íbamos a usar menos del 10%. La segunda es que el aspecto por defecto de Bootstrap no encajaba con la imagen de TECSOLED. Tailwind tampoco era una buena opción porque requería un paso de compilación que complicaba el despliegue.

El sistema CSS propio que desarrollamos, al que llamamos "Industrial Precisión", pesa menos de 15 KB, usa variables personalizadas para colores, tipografía y bordes, incluye modo oscuro nativo y nos dio control total sobre el diseño. La inversión de tiempo extra al principio merecía la pena.

Java.
Descripción de la herramienta.
Descripción del uso de la herramienta en el proyecto.

METODOLOGÍA

Metodología de trabajo

Usamos una metodología ágil adaptada a un proyecto de dos personas, con sprints de una o dos semanas. Al ser los dos de la misma empresa, nos reuníamos presencialmente para revisar el trabajo y planificar la siguiente iteración. El tablero de proyectos de GitHub nos sirvió para llevar el seguimiento y el historial de commits es el reflejo real del avance.

Reparto de trabajo

Desde el principio dividimos el trabajo por capas técnicas:

Michael se encargó del backend: configuración de Express, conexión a SQL Server y SQLite, implementación de los 9 endpoints, middlewares de autenticación JWT y modo MOCK para desarrollo sin conexión al ERP.

Javier se encargó del frontend: diseño del sistema CSS, maquetación de las 4 páginas, integración con la API, autocompletado, carga paralela con Promise.all e integración de Chart.js para el panel de facturación.

El análisis de requisitos, las pruebas y la documentación los hicimos entre los dos.

Planificación en fases

Fase	Actividades	Semanas	Horas (Michael)	Horas (Javier)
1. Análisis y diseño	Requisitos, identificación de tablas SQL Server, diseño de arquitectura.	Sem. 1-2	11	11
2. Backend	Express, SQL Server, SQLite, 9 endpoints, JWT, modo MOCK.	Sem. 3-4	40	5
3. Frontend	CSS propio, 4 páginas, integración con API, Chart.js.	Sem. 5-6	8	35
4. Pruebas y despliegue	Pruebas con Postman, corrección de errores, PM2, ajustes de seguridad.	Sem. 7-8	8	7
5. Documentacion	Memoria TFG, manuales de usuario e instalación.	Sem. 9-10	11	11

TOTAL	10 semanas	78 h	69 h

[INSERTAR: Diagrama de Gantt con las cinco fases]

Presupuesto

Concepto	Horas	Coste/hora	Total
Michael Gutierrez (backend + coord.)	78 h	20 euros/h	1.560 euros
Javier Arnedo (frontend + coord.)	69 h	20 euros/h	1.380 euros
Licencias de software	n/a	n/a	0 euros (todo software libre)
Servidor interno TECSOLED	n/a	n/a	0 euros (infraestructura existente)
TOTAL	147 h	20 euros/h	2.940 euros

README y control de versiones

El repositorio en GitHub incluye un README con instrucciones claras de instalación (clonar, npm install, configurar el .env y arrancar con PM2). Usamos commits con prefijos convencionales (feat, fix, docs, chore) para que el historial refleje fielmente el avance del proyecto y sirva de evidencia del trabajo realizado.

TRABAJOS FUTUROS

Pendiente para la entrega final (junio 2026)

Aún tenemos pendiente completar el despliegue en el servidor real de TECSOLED y verificar los tiempos con datos reales del ERP. También nos falta añadir las capturas de pantalla de la aplicación funcionando y redactar el manual de usuario para los empleados que no son técnicos.

Mejoras y ampliaciones que nos gustaría implementar

Una vez entregado el TFG, hay varias mejoras que ya tenemos en mente:

Alertas automáticas de stock: que la aplicación envié un aviso cuando el stock de algún artículo caiga por debajo del mínimo configurado en el ERP.

Exportación a Excel o CSV: poder descargar los resultados de una búsqueda o el informe del dashboard directamente desde la aplicación.

Tests automáticos: una suite de tests de integración para el backend con Jest y Supertest que verifique los 9 endpoints.

Acceso HTTPS fuera de la red local: con un certificado TLS y acceso por VPN para los empleados que trabajan desde casa.

Mas módulos del ERP: ampliar la aplicación con pedidos a proveedores, ficha de clientes o historial de ventas por artículo.

CONCLUSIONES

Valoración técnica

El objetivo que nos pusimos al principio, bajar de 5 segundos de tiempo de respuesta, se ha cumplido. Las mediciones que hicimos con Postman dan tiempos por debajo de 800 ms en todos los endpoints, incluidos los más pesados como la búsqueda con LIKE sobre la BD del ERP o las consultas agregadas de facturación.

La decisión de no usar frameworks en el frontend, que al principio generó alguna duda entre nosotros, creemos que fue la correcta. El resultado carga rápido, es fácil de entender y modificar, y el peso total de la página es mínimo. Para un proyecto de esta escala, añadir React o Vue hubiera sido sobre ingeniería.

Lo que más nos costó fue entender el esquema de la base de datos del ERP. No había documentación interna útil, así que Michael tuvo que ir explorando las tablas con el Query Analyzer hasta dar con las relaciones correctas. Esa parte llevó bastante más tiempo del previsto.

Aprendizajes

Para los dos este proyecto ha supuesto el primer desarrollo fullstack real de principio a fin. Lo que más hemos aprendido:

Michael: diseño de una API REST desde cero con autenticación JWT, consultas SQL sobre un esquema de BD que no disene yo, y gestión de la seguridad de una aplicación web (validación de entradas, helmet, bcrypt).

Javier: construir un sistema de diseño CSS escalable sin frameworks, integrar Chart.js para visualización de datos y manejar la comunicación asíncrona con Fetch API y Promise.all.

Valoración personal

Estamos satisfechos con el resultado. El hecho de que resuelva un problema real de la empresa donde trabaja Michael, y que algunos compañeros ya la estén probando en la fase de pruebas, le da un valor práctico que va más allá de lo académico. Ha sido el proyecto más exigente del ciclo, pero también el más motivador de los dos.

REFERENCIAS

OpenJS Foundation. (s.f.). Node.js Documentation. Recuperado de https://nodejs.org/en/docs

Documentacion oficial de Node.js. Consultada para la configuración del servidor y el manejo del event loop asíncrono. Aplicada en src/index.js y src/config/.

Express.js. (s.f.). Express 4.x API Reference. Recuperado de https://expressjs.com/en/4x/api.html

Referencia de Express 4.x. Consultada para la definición de rutas, middlewares encadenados y gestión de errores. Aplicada en src/routes/ y src/middlewares/.

Microsoft Corporation. (s.f.). mssql: Node.js driver for Microsoft SQL Server. Recuperado de https://www.npmjs.com/package/mssql

Documentacion del driver mssql. Consultada para la configuración del pool de conexiones y las queries parametrizadas. Aplicada en src/config/database.js y src/services/.

Auth0. (s.f.). Introduction to JSON Web Tokens. Recuperado de https://jwt.io/introduction

Guía sobre el estándar JWT. Consultada para el diseño del sistema de autenticación y la estructura del payload. Aplicada en src/middlewares/auth.middleware.js.

npm, Inc. (s.f.). bcryptjs. Recuperado de https://www.npmjs.com/package/bcryptjs

Documentacion de bcryptjs. Consultada para el hash seguro de contraseñas con salt. Aplicada en src/controllers/auth.controller.js.

Chart.js Contributors. (s.f.). Chart.js Documentation v4. Recuperado de https://www.chartjs.org/docs/latest

Documentacion de Chart.js 4.x. Consultada para la configuración del gráfico de barras del panel de facturación. Aplicada en js/dashboard.js.

Mozilla Developer Network. (s.f.). MDN Web Docs: Web technology for developers. Recuperado de https://developer.mozilla.org

Referencia principal del frontend: Fetch API, CSS custom properties, Promise.all y sessionStorage. Aplicada en js/ y css/styles.css.

WillowTree, Inc. (s.f.). better-sqlite3. Recuperado de https://github.com/WillowTree/better-sqlite3

Documentacion del driver SQLite para Node.js. Consultada para la gestión de la base de datos de usuarios. Aplicada en src/config/database-usuarios.js.

Keymetrics. (s.f.). PM2: Advanced Production Process Manager. Recuperado de https://pm2.keymetrics.io/docs

Documentacion de PM2. Consultada para la configuración del arranque automático y la gestión de logs en producción. Aplicada en ecosystem.config.js.

OWASP Foundation. (2021). OWASP Top Ten 2021. Recuperado de https://owasp.org/www-project-top-ten

Lista de los diez riesgos más críticos en aplicaciones web. Consultada para las decisiones de seguridad del backend: validación de entradas, gestión de tokens y cabeceras HTTP.

helmetjs. (s.f.). Helmet.js Documentation. Recuperado de https://helmetjs.github.io

Documentacion de Helmet. Consultada para configurar las cabeceras de seguridad HTTP del servidor Express. Aplicada en src/index.js.